# LAPORAN PRAKTIKUM

# MODUL 7 IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan praktikum

Mata Kuliah Teknologi IOT II




Di Susun Oleh:

**NAMA : M. ROMZI**

**NPM : 25782015**

**KELAS : 3A**

**PRODI : TRI**

**DOSEN PENGAMPU :**

**Dr. Ir. Septafiansyah Dwi Putra, S. T., M.T.**

**Agus Ambarwari, S.Pd.,M.Kom.**

## DINAS PENDIDIKAN DAN KEBUDYAAN

## KOTA BANDAR LAMPUNG

## POLITEKNIK NEGERI LAMPUNG

## TAHUN AJARAN 2026/2027

## Praktikum 1 : Pembangunan Ekosistem IoT Server dengan Docker Compose

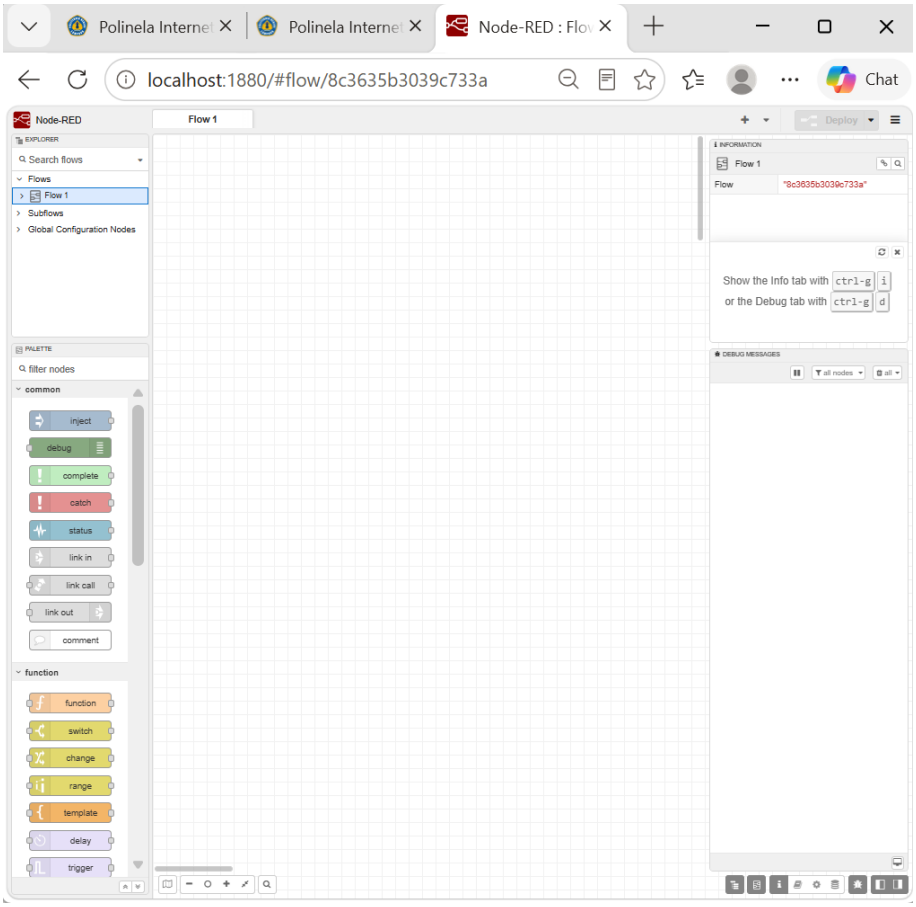
Pada praktikum ini, kita akan membangun infrastruktur server IoT lokal terpadu (Mosquitto MQTT Broker dan Node-RED) secara otomatis hanya menggunakan berkas deklaratif Docker Compose.

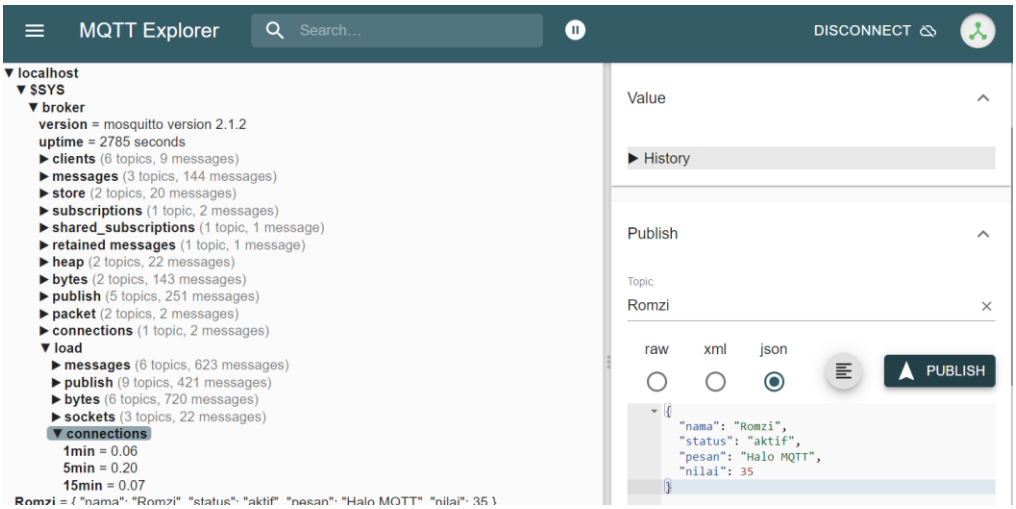
```
C:\Users\ROMZI\docker\iot-server2>docker compose up -d
[+] up 29/29
 ✔Image nodered/node-red:latest  Pulled                                    98.7s
 ✔Image eclipse-mosquitto:latest Pulled                                    41.5s
 ✔Network iot-server2_default    Created                                    0.1s
 ✔Container mqtt-broker          Started                                    1.4s
 ✔Container node-red             Started                                    1.6s

C:\Users\ROMZI\docker\iot-server2>
```

```
C:\Users\ROMZI\docker\iot-server2>docker compose up -d
[+] up 3/3
 ✔Network iot-server2_default Created                    0.1s
 ✔Container mqtt-broker       Started                    0.9s
 ✔Container node-red          Started                    1.1s

C:\Users\ROMZI\docker\iot-server2>docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED          STATUS                    PORTS
                                                                        NAMES
5805316bc901   nodered/node-red:latest        "./entrypoint.sh"        41 seconds ago   Up 40 seconds (healthy)   0.0.0.0:1880->1880/tcp,
 [::]:1880->1880/tcp                                                    node-red
8a874720ee47   eclipse-mosquitto:latest       "/docker-entrypoint.…"   41 seconds ago   Up 40 seconds             0.0.0.0:1883->1883/tcp,
 [::]:1883->1883/tcp, 0.0.0.0:9001->9001/tcp, [::]:9001->9001/tcp       mqtt-broker
```





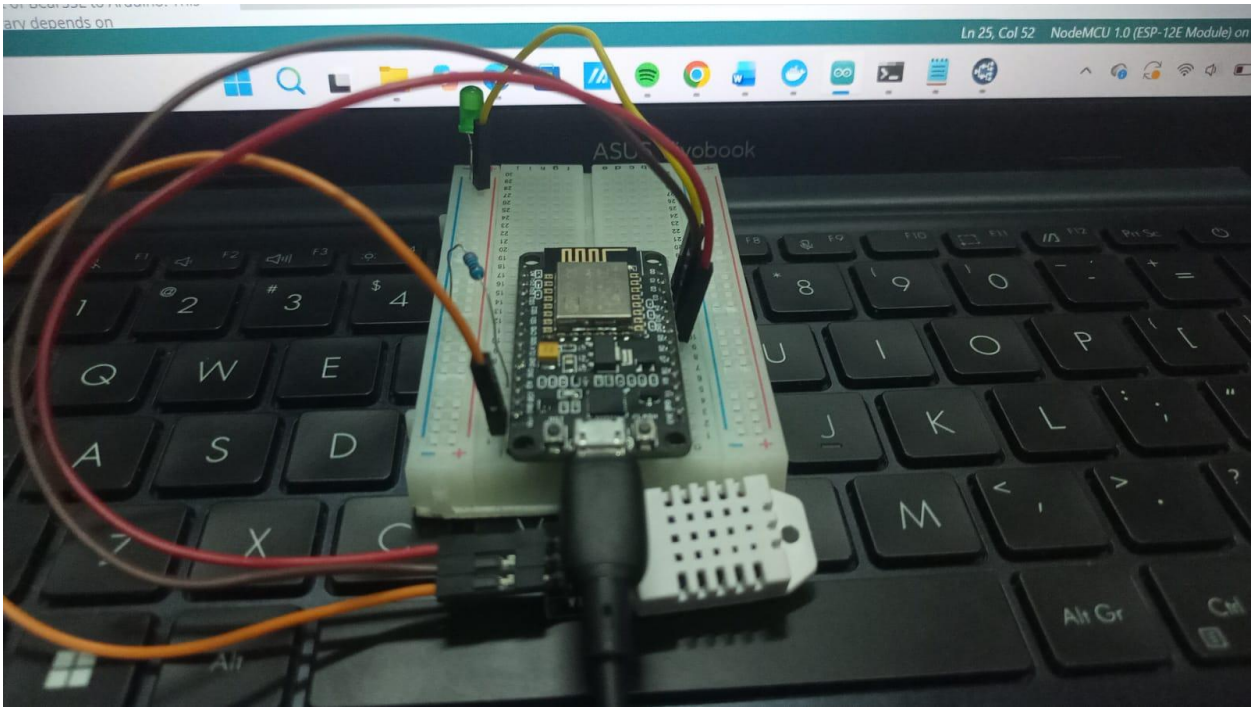

### 7.5.2 Praktikum 2: Migrasi Simpul NodeMCU ESP8266 ke Broker MQTT Lokal

Setelah server lokal berjalan di laptop, langkah berikutnya adalah mengarahkan NodeMCU ESP8266 dari broker publik ke alamat IP lokal laptop Anda dalam jaringan Wi-Fi lokal.

**Skema Rangkaian Perangkat Keras:**

Rangkaian perangkat keras pada praktikum ini identik dengan konfigurasi pada Modul 6 (hanya menggunakan sensor DHT dan indikator LED):

- **Sensor DHT11/DHT22:**
    - Pin VCC →Pin 3V3 NodeMCU
    - Pin GND →Pin GND NodeMCU
    - Pin Data →Pin **D4 (GPIO 2)**
- **LED Indikator Aktuator:**
    - Anoda (kaki panjang) →Pin **D1 (GPIO 5)**
    - Katoda (kaki pendek) →Resistor 220 Ω →Pin GND




sketch_oct12a_.ino

```
 * PRAKTIKUM 7: SIMPUL SENSOR-AKTUATOR IOT UNTUK BROKER LOKAL
 * ============================================================================
 * Program ini membaca sensor DHT11/22, menyusun paket telemetri JSON, mengirimkannya
 * ke Broker Mosquitto Lokal di Laptop, serta mendengarkan instruksi kendali LED.
 * ============================================================================
 */

#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
const char* ssid     = "cihuyyy";
const char* password = "12345678";

// 2. Konfigurasi Broker MQTT Lokal
```

Output Serial Monitor ×

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3') New Line 1152

```
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":1.70,"kelembapan":16.20,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":1.70,"kelembapan":15.80,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":1.70,"kelembapan":15.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":1.70,"kelembapan":15.70,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":1.60,"kelembapan":15.80,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":1.60,"kelembapan":15.90,"status_led":0}
```

### 7.5.3 Praktikum 3: Integrasi Alur Data Telemetri Dua Arah pada Node-RED

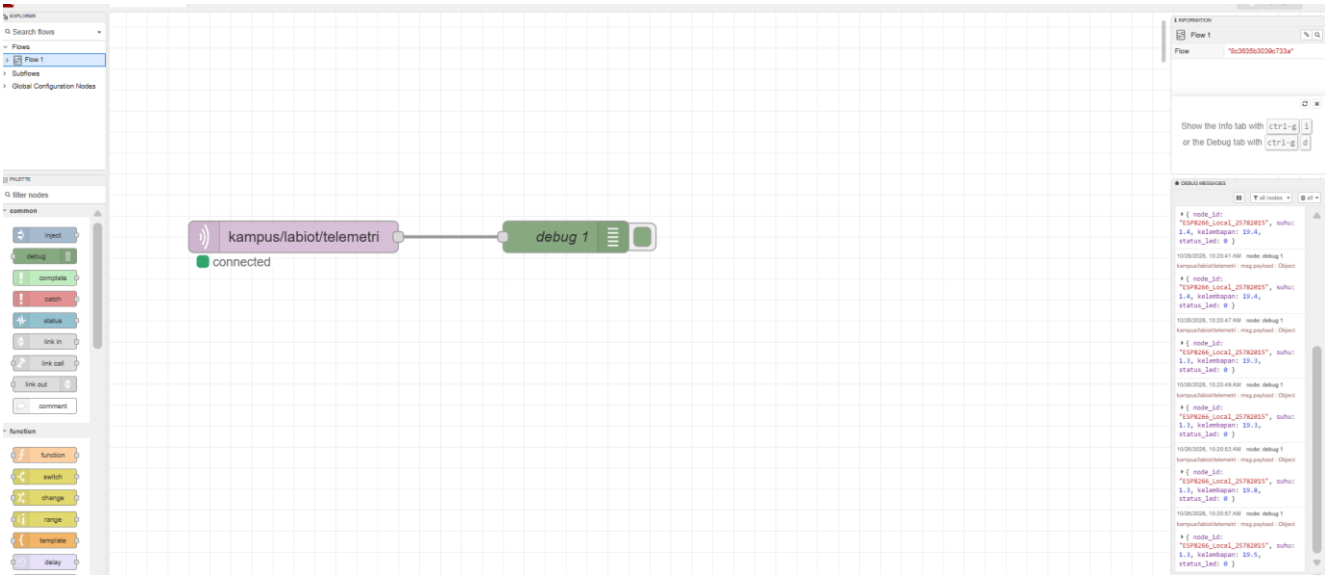
Pada praktikum ini, kita akan menangkap dan membedah aliran data telemetri yang dikirimkan oleh NodeMCU menggunakan antarmuka kanvas Node-RED.

**Langkah Kerja:**

1. Buka peramban web dan akses antarmuka Node-RED di **http://localhost:1880**.
2. **Menambahkan Konfigurasi Server MQTT Broker:**
    - Di sebelah kiri (*Palette*), temukan kategori **network**, lalu klik dan seret (*drag*) node **mqtt in** ke tengah kanvas kerja.
    - Klik ganda (*double-click*) pada node mqtt in tersebut untuk membuka panel pengaturannya:
        - Pada baris **Server**, klik ikon pensil ( ) untuk membuat konfigurasi broker baru.
        - **Name:** Mosquitto Local
        - **Server:** Ketik **mqtt**
          *(PENTING: Jangan gunakan localhost atau IP laptop! Karena Node-RED dan Mosquitto berada dalam satu jaringan Docker Compose yang sama, kontainer Node-RED dapat memanggil broker secara langsung menggunakan nama servis deklaratifnya, yaitu mqtt).*
        - **Port:** 1883
        - Klik tombol merah **Add** di pojok kanan atas dialog.
3. **Mengonfigurasi Parameter Topik:**
    - Pada baris **Topic**, masukkan: kampus/labiot/telemetri
    - Pada baris **Output**, ubah dari *auto-detect* menjadi **a parsed JSON object**. (Fitur ini secara otomatis mengurai teks string JSON menjadi objek JavaScript hierarkis tanpa perlu menulis fungsi *parser* manual).
    - Klik tombol **Done**.
4. **Menambahkan Node Pemantau (*Debug Node*):**
    - Pada *Palette* sebelah kiri di kategori **common**, seret node **debug** ke kanvas.
    - Tarik garis (*wire*) dari titik port keluaran di sisi kanan node mqtt in ke port masukan di sisi kiri node debug.
5. **Menerapkan Alur Kerja (*Deploy*):**

- Klik tombol merah **Deploy** di sudut kanan atas antarmuka.
- Amati bahwa di bawah node mqtt in muncul teks indikator hijau bertuliskan **connected**.

6. **Menganalisis Aliran Data:**
   - Klik tab ikon kumbang ( 🐞 ) di bilah sisi kanan (*Sidebar*) untuk membuka panel **Debug Messages**.
   - Anda akan melihat objek JSON masuk setiap 4 detik secara *real-time*:
   - kampus/labiot/telemetri : msg.payload : Object
   - {
   - node_id: "ESP8266_Local_Node01",
   - suhu: 29.8,
   - kelembapan: 62.4,
   - status_led: 0
   - }
   - Klik tanda panah segitiga kecil di samping teks Object untuk mengeksplorasi hierarki pasangan kunci dan nilai data.



### 7.5.4 Praktikum 4: Implementasi Rule Engine Otomasi Aktuator Loop Tertutup

Pada praktikum ini, kita akan membangun *Closed-Loop Rule Engine* otonom:

- **JIKA** nilai suhu yang dibaca oleh sensor melebihi ambang batas kritis 31.0°C: Node-RED secara otomatis memublikasikan paket JSON {"led": 1} untuk menyalakan LED indikator peringatan bahaya di pin D1.
- **JIKA** suhu berada dalam batas normal (≤ 31.0°C): Node-RED memublikasikan paket JSON {"led": 0} untuk memadamkan LED indikator.

**Langkah Kerja:**

1. **Membuat Node Percabangan Logika (switch):**
    - Pada kategori **function**, seret node **switch** ke kanvas.
    - Hubungkan port keluaran node mqtt in ke port masukan node switch.
    - Klik ganda node switch untuk menyetel aturan logika evaluasi:
        - **Property:** Ganti msg.payload menjadi **msg.payload.suhu**.
        - **Aturan 1 (Kondisi Panas):** Pilih operator **>**, lalu masukkan angka: **31**.
        - Klik tombol **+ add** di bagian bawah untuk menambahkan aturan kedua.
        - **Aturan 2 (Kondisi Aman):** Pilih operator **<=**, lalu masukkan angka: **31**.
        - Klik **Done**. Perhatikan bahwa node switch kini memiliki dua port keluaran bernomor 1 (atas) dan 2 (bawah).
2. **Membuat Node Transformasi Muatan Panas (change):**
    - Seret node **change** dari kategori *function* ke kanvas di sebelah kanan atas node switch.
    - Hubungkan port keluaran 1 (suhu > 31) dari node switch ke masukan node change pertama ini.
    - Klik ganda node change:
        - **Name:** Nyalakan LED Indikator
        - **Set:** msg.payload
        - **To:** Ubah tipe data dari string (az) menjadi **JSON** ({ }), lalu masukkan nilai:
        - {

- "led": 1

}

- Klik **Done**.

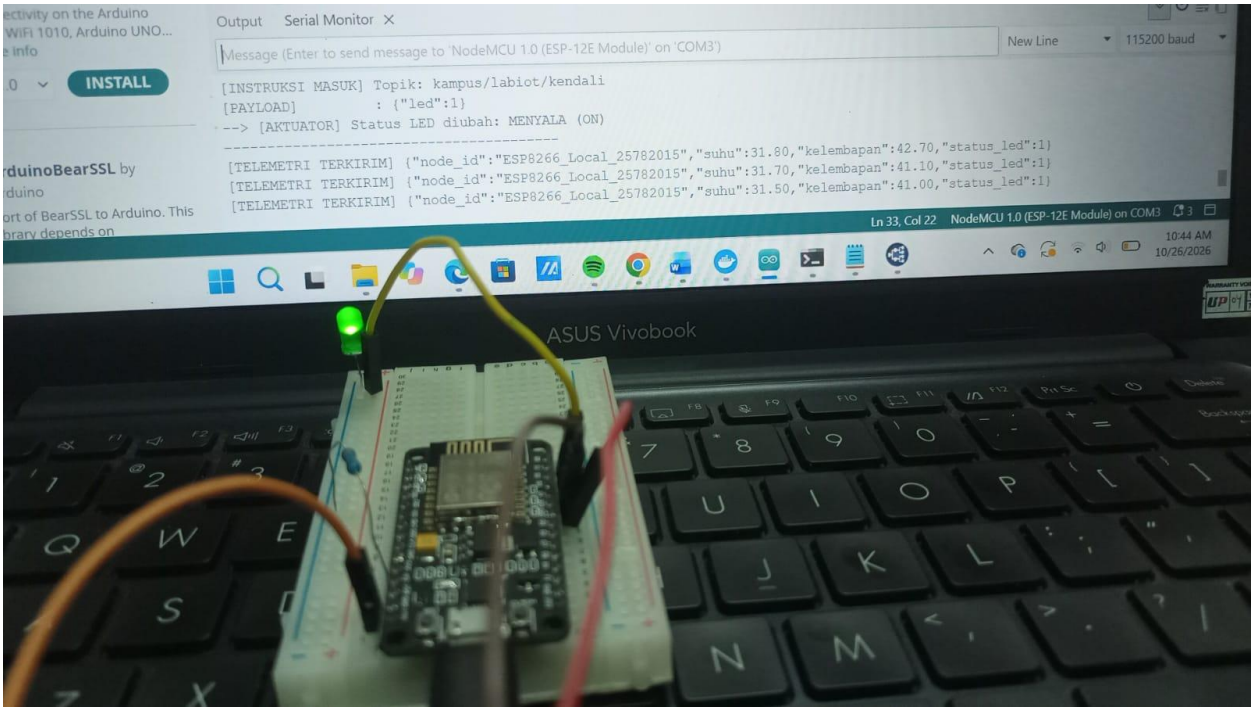
3. **Membuat Node Transformasi Muatan Normal (change):**
    - Seret node **change** kedua ke kanvas di sebelah kanan bawah node switch.
    - Hubungkan port keluaran 2 (suhu $\leq 31$) dari node switch ke masukan node change kedua ini.
    - Klik ganda node change:
        - **Name:** Padamkan LED Indikator
        - **Set:** msg.payload
        - **To:** Ubah tipe data menjadi **JSON** ({ }), lalu masukkan nilai:
        - {
        - "led": 0

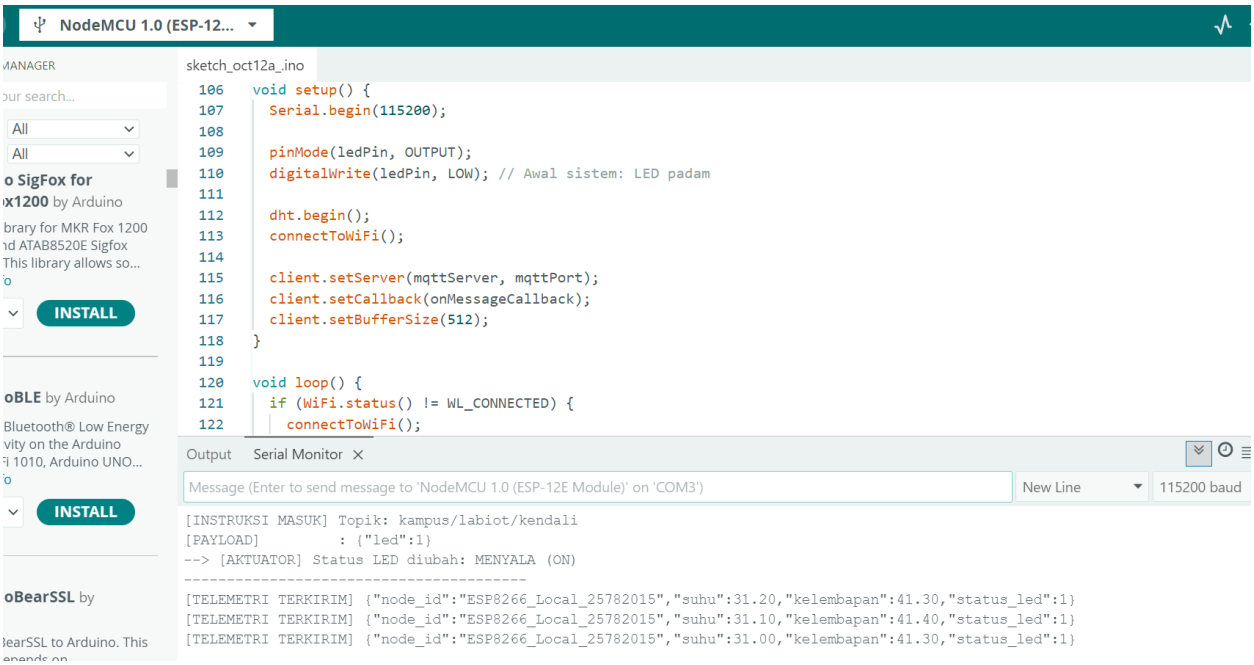
}

- Klik **Done**.

4. **Menerbitkan Instruksi Balik ke Broker (mqtt out):**
    - Pada kategori **network**, seret node **mqtt out** ke kanvas.
    - Hubungkan keluaran dari node change pertama dan node change kedua secara bersamaan ke port masukan node mqtt out.
    - Klik ganda node mqtt out:
        - **Server:** Pilih Mosquitto Local.
        - **Topic:** kampus/labiot/kendali
        - **QoS:** 0 (atau 1)
        - **Retain:** false
        - Klik **Done**.
5. **Menerapkan Perubahan (*Deploy*):**
    - Klik tombol merah **Deploy** di kanan atas antarmuka.
6. **Pengujian Fisik Sistem Otomasi Tertutup:**

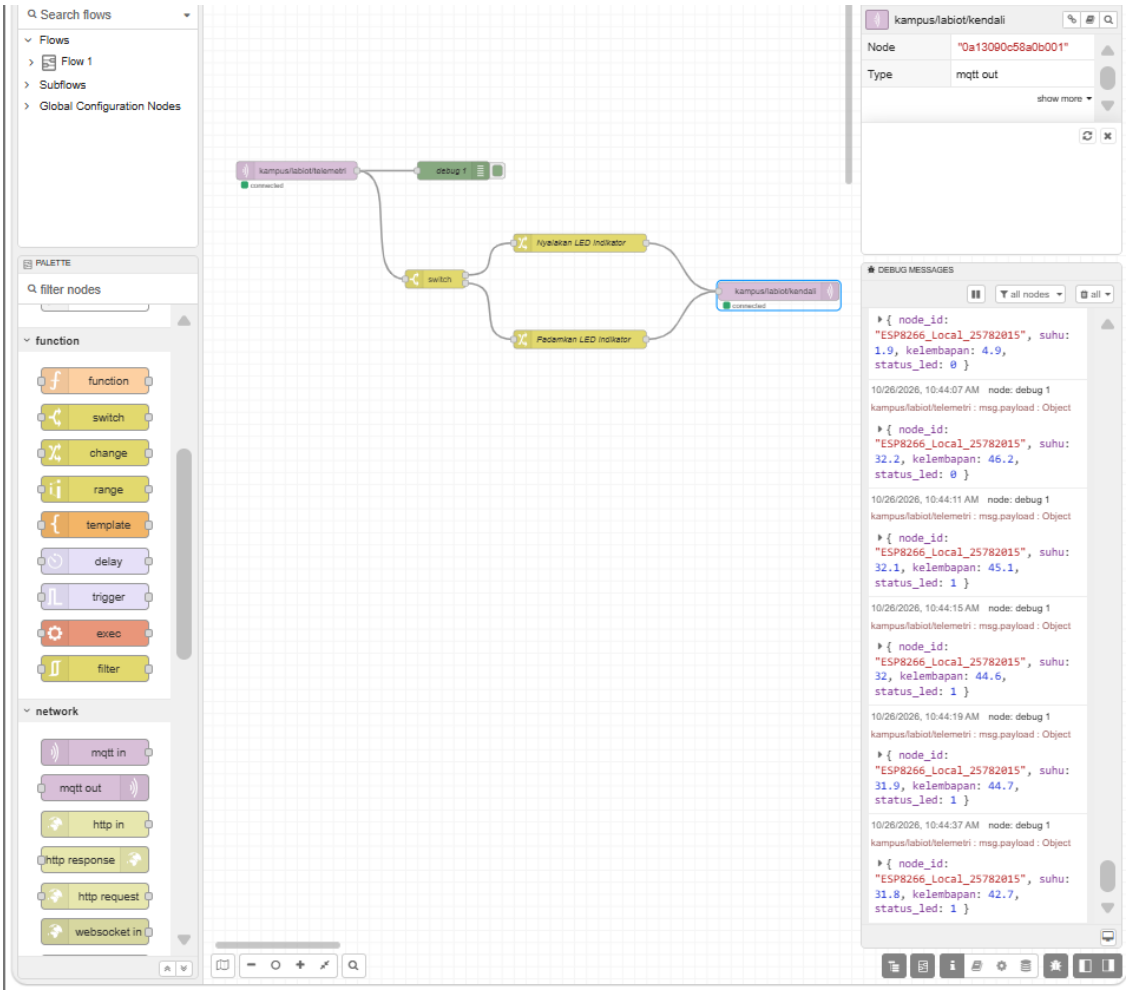
- Amati status awal: Karena suhu ruangan umumnya berkisar antara 26°C – 29°C(≤ 31°C), LED indikator berada dalam kondisi padam.
- Dekatkan sumber panas ringan (seperti ujung jari Anda yang ditempelkan kuat pada sensor DHT, atau hembusan napas hangat) hingga pembacaan suhu pada panel Debug merangkak naik melewati 31.0°C.
- **Hasil Pengamatan:** Seketika suhu melampaui 31.0°C, Node-RED mengeksekusi port 1, mengirimkan payload {"led": 1}, dan LED indikator bahaya di pin D1 seketika menyala terang!
- Lepaskan jari Anda dan biarkan suhu mendingin kembali ke bawah 31.0°C. LED indikator akan padam kembali secara otomatis.






```
106  void setup() {
107    Serial.begin(115200);
108
109    pinMode(ledPin, OUTPUT);
110    digitalWrite(ledPin, LOW); // Awal sistem: LED padam
111
112    dht.begin();
113    connectToWiFi();
114
115    client.setServer(mqttServer, mqttPort);
116    client.setCallback(onMessageCallback);
117    client.setBufferSize(512);
118  }
119
120  void loop() {
121    if (WiFi.status() != WL_CONNECTED) {
122      connectToWiFi();
```

```
[INSTRUKSI MASUK] Topik: kampus/labiot/kendali
[PAYLOAD]         : {"led":1}
--> [AKTUATOR] Status LED diubah: MENYALA (ON)
----------------------------------------
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":31.20,"kelembapan":41.30,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":31.10,"kelembapan":41.40,"status_led":1}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":31.00,"kelembapan":41.30,"status_led":1}
```



## 7.6 Latihan

- **Latihan 1 (Analisis Jaringan Internal Docker & Service Discovery):**
  Pada Praktikum 3, di dalam konfigurasi node mqtt in pada Node-RED, kita mengisi alamat server broker dengan nama teks **mqtt**, bukan alamat IP lokal laptop (192.168.1.15) maupun localhost.

i. Mengapa penulisan localhost pada Node-RED di dalam kontainer Docker akan menyebabkan kegagalan koneksi (*Connection Refused*)?
**Jawaban:** Karena setiap kontainer Docker memiliki *network namespace* (lingkungan jaringan) yang terisolasi. Jika Node-RED memanggil localhost (127.0.0.1), sistem akan mencarinya di dalam kontainer Node-RED itu sendiri, bukan di laptop *host* ataupun kontainer Mosquitto. Karena broker Mosquitto berada di kontainer terpisah (mqtt-broker), tidak ada servis yang aktif di port 1883 pada *loopback* kontainer Node-RED, sehingga koneksi ditolak.

ii. Jelaskan bagaimana mekanisme *Docker Embedded DNS Server* dan *Bridge Network* bekerja sehingga kontainer node-red dapat menemukan kontainer mqtt-broker cukup dengan memanggil nama layanannya!
**Jawaban:** Saat kontainer-kontainer digabungkan dalam satu jaringan *bridge* (melalui docker-compose.yml), Docker mengaktifkan peladen DNS internal (*Docker Embedded DNS Server*). Ketika Node-RED mencoba menghubungi hostname mqtt, DNS server internal Docker akan otomatis menerjemahkannya menjadi alamat IP kontainer mqtt-broker yang valid di jaringan tersebut. Proses *Service Discovery* ini membuat komunikasi antar-kontainer tetap stabil tanpa harus menggunakan IP statis laptop yang bisa berubah-ubah.

- **Latihan 2 (Penanganan Masalah Konektivitas Jaringan & Subnet Firewall):**
  Seorang mahasiswa mencoba menghubungkan NodeMCU ESP8266 ke broker lokal di laptopnya. ESP8266 berhasil terhubung ke Wi-Fi dan mendapatkan alamat IP 192.168.43.102, sedangkan laptop memiliki alamat IP 192.168.1.15.

i. Mengapa NodeMCU tidak akan pernah berhasil memublikasikan data ke broker Mosquitto pada kondisi tersebut?
**Jawaban:** NodeMCU dan laptop berada pada subnet jaringan yang berbeda (misalnya NodeMCU di 192.168.43.x dari *hotspot* HP, sedangkan laptop di 192.168.1.x dari Wi-Fi rumah). Karena berada di domain *broadcast* yang berbeda, paket TCP/IP tidak dapat merutekan komunikasi secara langsung tanpa adanya *router* atau aturan *port forwarding* khusus.

ii. Langkah teknis apa yang harus dilakukan mahasiswa tersebut agar kedua perangkat berada dalam domain *broadcast / subnet* yang sama?

**Jawaban:**

- Hubungkan laptop ke sumber jaringan (Wi-Fi / *Tethering* ponsel) yang sama persis dengan yang digunakan oleh NodeMCU.
- Atau, hubungkan kedua perangkat ke *router* fisik lokal yang sama agar mendapatkan alokasi IP dalam satu kelas/subnet yang seragam.

- **Latihan 3 (Implementasi Filter Histeresis pada Rule Engine):**
  Pada sistem kendali dua posisi (*Bang-Bang*), jika suhu berfluktuasi cepat di sekitar ambang batas 31.0°C(misal: 31.05°C → 30.95°C → 31.02°C), lampu LED indikator akan berkedip-kedip cepat (*flickering / hunting*) yang mengganggu stabilitas visual sistem.
  Rancang modifikasi alur Node-RED menggunakan **function node** berbahasa JavaScript yang menerapkan logika **Histeresis**:
  - LED baru menyala jika suhu $> 31.5$°C(msg.payload = {"led": 1}).
  - LED baru padam jika suhu mendingin hingga $< 30.0$°C(msg.payload = {"led": 0}).
    Tuliskan kode JavaScript yang Anda masukkan ke dalam *Function Node* tersebut!

```
let ledState = context.get('ledState') || 0;

let suhu = msg.payload.suhu;


// Logika Histeresis

if (suhu > 31.5) {

   ledState = 1; // Nyalakan LED jika suhu melebihi batas atas

} else if (suhu < 30.0) {

   ledState = 0; // Matikan LED jika suhu turun di bawah batas bawah

}

// Jika suhu di antara 30.0 - 31.5°C, status LED dipertahankan


context.set('ledState', ledState);

msg.payload = { "led": ledState };

return msg;
```

- **Latihan 4 (Analisis Mekanisme Volume Mount & Persistensi Data):**
  Pada berkas docker-compose.yml, terdapat pemetaan direktori - ./nodered:/data dan - ./mosquitto/data:/mosquitto/data.

i. Apa yang akan terjadi pada alur kanvas visual Node-RED yang telah Anda buat jika kontainer dimatikan dengan perintah docker compose down, kemudian dinyalakan kembali, seandainya baris volume tersebut dihapus dari docker-compose.yml?
**Jawaban:** Seluruh alur kerja (*flows*), node, dan konfigurasi yang telah dibuat di Node-RED akan hilang total. Penyimpanan internal kontainer bersifat sementara (*ephemeral*). Tanpa adanya *Volume Mount* yang memetakan data ke penyimpanan *host*, penghapusan kontainer juga akan memusnahkan seluruh data di dalamnya, sehingga Node-RED akan kembali seperti baru saat dinyalakan ulang.

ii. Jelaskan perbedaan mendasar antara *Bind Mount* dan *Named Volume* pada Docker!
**Jawaban:**

- ***Bind Mount*:** Memetakan folder atau berkas yang ada di komputer *host* secara langsung ke dalam kontainer. Sangat cocok untuk pengembangan (*development*) karena kita bisa menyunting berkas secara langsung dari laptop.
- ***Named Volume*:** Area penyimpanan khusus yang dibuat dan dikelola sepenuhnya oleh Docker di dalam sistem internalnya (terisolasi dari direktori kerja utama *host*). Lebih disarankan untuk lingkungan produksi (*production*) karena tingkat keamanan dan pengelolaan datanya lebih terstruktur oleh Docker.

## 7.7 Tugas: Pengamanan Server IoT Industri (MQTT Authentication & Access Control)

Dalam dunia industri nyata, membiarkan MQTT Broker beroperasi tanpa kata sandi (allow_anonymous true) adalah pelanggaran fatal terhadap standar keamanan siber (*Cybersecurity Vulnerability*). Pada tugas ini, kembangkan ekosistem server IoT lokal Anda menjadi sistem yang terproteksi penuh menggunakan sistem autentikasi kredensial (*Username & Password*).

**Instruksi Pengerjaan Langkah demi Langkah:**

**1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:**
Buka terminal/PowerShell di laptop Anda, lalu jalankan perintah berikut untuk mengeksekusi utilitas mosquitto_passwd langsung di dalam kontainer Mosquitto yang sedang berjalan:
docker exec -it mqtt-broker mosquitto_passwd -c /mosquitto/config/password.txt admin

- Sistem akan meminta Anda memasukkan kata sandi. Masukkan kata sandi: **rahasia123** (teks tidak akan muncul di layar saat diketik), lalu tekan Enter dan konfirmasi ulang kata sandi tersebut.
- Buka folder iot-server/mosquitto/config/ pada laptop Anda, pastikan kini terdapat berkas baru bernama password.txt yang berisi baris *hash* kredensial akun admin.

**2. Memperbarui Konfigurasi Kebijakan Mosquitto:**

Buka berkas iot-server/mosquitto/config/mosquitto.conf di editor teks Anda:

- Ubah nilai allow_anonymous true menjadi:
  allow_anonymous false
- Tambahkan baris pemanggilan berkas kata sandi:
  password_file /mosquitto/config/password.txt
- Simpan berkas mosquitto.conf.

**3. Memuat Ulang Layanan Broker:**

Restart kontainer mqtt-broker melalui terminal agar kebijakan keamanan baru diterapkan:
docker restart mqtt-broker

**4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):**

- Buka antarmuka Node-RED di browser: Amati bahwa di bawah node mqtt in dan mqtt out, indikator status kini berubah menjadi warna merah bertuliskan **disconnected** atau **connection refused**.
- Buka Serial Monitor NodeMCU: Amati bahwa koneksi mikrokontroler kini gagal dan berulang kali menampilkan kode penolakan:
- [MQTT] Menghubungkan ke Broker Lokal... Gagal (rc=5). Coba lagi dalam 4 detik...
  *(Catatan: Kode status rc=5 merupakan representasi standar MQTT untuk 'Connection Refused: Not Authorized').*

**5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):**

1. **Pemutakhiran Skrip Program NodeMCU ESP8266:**
   - Buka sketsa program Arduino IDE dari Praktikum 2.
   - Modifikasi bagian pemanggilan fungsi client.connect() di dalam fungsi reconnectMQTT() dengan menyertakan kredensial resmi:
   - // Format: client.connect(clientId, username, password)
   - if (client.connect(clientId, "admin", "rahasia123")) {
   - Serial.println(" Terhubung dengan Autentikasi Sah!");
   - client.subscribe(topicSubCmd);

   }
   - Unggah ulang kode ke NodeMCU dan amati bahwa ESP8266 kini kembali berhasil terhubung (*rc=0*).
2. **Pemutakhiran Konfigurasi Kredensial pada Node-RED:**
   - Pada kanvas Node-RED, klik ganda node mqtt in.
   - Klik ikon pensil ( ) di samping nama broker Mosquitto Local.
   - Buka tab **Security** di bagian atas dialog konfigurasi.

- Masukkan **Username:** admin dan **Password:** rahasia123.
- Klik tombol **Update**, lalu klik **Done**, dan tekan tombol merah **Deploy**.
- Amati bahwa indikator node MQTT pada kanvas Node-RED seketika kembali berwarna hijau (**connected**), dan aliran data telemetri serta kendali otomasi indikator LED loop tertutup kembali berfungsi normal dan aman dari gangguan pihak luar!

**1. Membuat Berkas Kredensial Terenkripsi di Dalam Kontainer:**

Buka terminal/PowerShell di laptop Anda, lalu jalankan perintah berikut untuk mengeksekusi utilitas mosquitto_passwd langsung di dalam kontainer Mosquitto yang sedang berjalan:

```
C:\Users\ROMZI\docker\iot-server2>docker exec -it mqtt-broker mosqu
itto_passwd -c /mosquitto/config/password.txt admin
Password:
Reenter password:
Adding password for user admin
```

**2. Memperbarui Konfigurasi Kebijakan Mosquitto:**

Buka berkas iot-server/mosquitto/config/mosquitto.conf di editor teks Anda:

```
listener 1883
protocol mqtt

listener 9001
protocol websockets

allow_anonymous false
password_file /mosquitto/config/password.txt

persistence true
persistence_location /mosquitto/data/

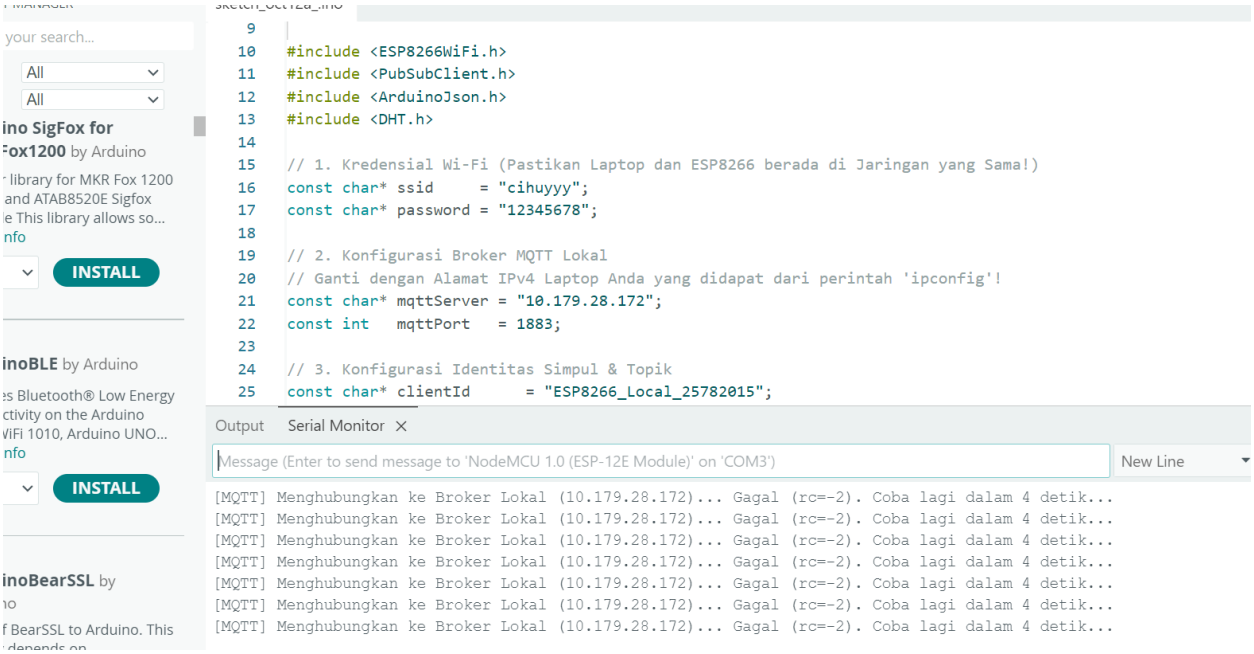
log_dest file /mosquitto/log/mosquitto.log
log_dest stdout
```

**3. Memuat Ulang Layanan Broker:**

Restart kontainer mqtt-broker melalui terminal agar kebijakan keamanan baru diterapkan:

```
C:\Users\ROMZI\docker\iot-server2>docker restart mqtt-broker
mqtt-broker
```

## 4. Analisis Dampak Kegagalan Sistem (Observasi Masalah):



```
 9
10  #include <ESP8266WiFi.h>
11  #include <PubSubClient.h>
12  #include <ArduinoJson.h>
13  #include <DHT.h>
14
15  // 1. Kredensial Wi-Fi (Pastikan Laptop dan ESP8266 berada di Jaringan yang Sama!)
16  const char* ssid     = "cihuyyy";
17  const char* password = "12345678";
18
19  // 2. Konfigurasi Broker MQTT Lokal
20  // Ganti dengan Alamat IPv4 Laptop Anda yang didapat dari perintah 'ipconfig'!
21  const char* mqttServer = "10.179.28.172";
22  const int   mqttPort   = 1883;
23
24  // 3. Konfigurasi Identitas Simpul & Topik
25  const char* clientId      = "ESP8266_Local_25782015";
```

```
Output  Serial Monitor ×
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')   New Line
[MQTT] Menghubungkan ke Broker Lokal (10.179.28.172)... Gagal (rc=-2). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (10.179.28.172)... Gagal (rc=-2). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (10.179.28.172)... Gagal (rc=-2). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (10.179.28.172)... Gagal (rc=-2). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (10.179.28.172)... Gagal (rc=-2). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (10.179.28.172)... Gagal (rc=-2). Coba lagi dalam 4 detik...
[MQTT] Menghubungkan ke Broker Lokal (10.179.28.172)... Gagal (rc=-2). Coba lagi dalam 4 detik...
```

## 5. Rekayasa Perbaikan Sistem (Penyelesaian Tugas):

## 1. Pemutakhiran Skrip Program NodeMCU ESP8266



```
87
88  void reconnectMQTT() {
89    while (!client.connected()) {
90      Serial.print("[MQTT] Menghubungkan ke Broker Lokal (");
91      Serial.print(mqttServer);
92      Serial.print(")...");
93
94      // Format: client.connect(clientId, username, password)
95      if (client.connect(clientId, "admin", "rahasia123")) {
96        Serial.println(" Terhubung dengan Autentikasi Sah!");
97        // Segera daftarkan subskripsi topik kendali
98        client.subscribe(topicSubCmd);
99        Serial.printf("[MQTT] Berlangganan ke topik: %s\n", topicSubCmd);
100     } else {
101       Serial.printf(" Gagal (rc=%d). Coba lagi dalam 4 detik...\n", client.state());
102       delay(4000);
103     }
```

```
Output  Serial Monitor ×
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')   New Line   115200 baud
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":27.50,"kelembapan":43.00,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":27.50,"kelembapan":43.20,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":27.50,"kelembapan":43.20,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":27.50,"kelembapan":43.60,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":27.40,"kelembapan":44.20,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":27.50,"kelembapan":44.40,"status_led":0}
[TELEMETRI TERKIRIM] {"node_id":"ESP8266_Local_25782015","suhu":27.50,"kelembapan":44.30,"status_led":0}
```

## 2.Pemutakhiran Konfigurasi Kredensial pada Node-RED

